American International University–Bangladesh (AIUB)
Department of Computer Science, Faculty of Science and Technology

Mid-Term Examination Suggestions

Faculty: Mejbah Ahammad **Email:** ahammadmejbah@aiub.edu
Semester: Summer 2025-2026 **Date: July 25, 2026**

Exam Pattern

Part	Marks	Question Style
Part A	15	OBE (Full Industry Scenario)
Part B	30	20 MCQs (20×1.5)
Part C	15	Board Question (Full Industry Scenario)

1 Lecture 1 – Software & Software Engineering

1.1 OBE Scenario Suggestions

1. **Netflix Case Study**

   Netflix is modernizing its global streaming platform to improve recommendation quality, reduce service interruptions and support millions of simultaneous users. However, the development team repeatedly misses deadlines, exceeds the allocated budget and receives increasing customer complaints regarding poor performance and system failures. Different departments continuously request new features after development has started. As a software engineer, analyze the causes of project failure, identify the software crisis reflected in this scenario, recommend suitable software maintenance activities and explain why applying software engineering principles is essential for delivering high-quality software.

2. **Bangladesh Railway Case Study**

   Bangladesh Railway introduces a nationwide online ticketing system. Shortly after deployment, new government regulations require additional security features, users request interface improvements and several software bugs are reported. Analyze the situation and determine which software maintenance activities should be performed for each issue. Justify your answers with appropriate software engineering concepts.

1.2 Board Question Suggestions

1. A government agency plans to develop a National Digital Identity System for millions of citizens. Discuss the characteristics of software, explain each phase of the Software Development Life Cycle (SDLC), identify software myths that may negatively affect the project and discuss possible reasons for software failure.

2. Compare the characteristics of good software and bad software using a real-world industrial example.

1.3 MCQ Focus

- Software Engineering
- Software Crisis
- Software Characteristics
- Software Applications
- SDLC
- Software Maintenance
- Software Myths
- Software Failure

Mejbah Ahammad, Lecturer, Department of Data Science, American International University-Bangladesh (AIUB), ahammadmejbah@aiub.edu

2 Lecture 2 – Software Development Process Models

2.1 OBE Scenario Suggestions

1. **Tesla Autonomous Driving System**

   Tesla is developing autonomous driving software where passenger safety is the highest priority. System requirements are clearly defined before development begins, and every development phase must have corresponding verification and validation activities. Select the most suitable SDLC model for this project. Justify your selection by discussing the characteristics of the chosen model and explain why alternative models are less suitable.

2. **Airbnb AI Travel Assistant**

   Airbnb plans to develop an AI-powered travel assistant. Customer requirements change almost every week because users continuously suggest new features after testing early versions. The company wants to release working software quickly while continuously improving it based on customer feedback. Select the most appropriate software process model and justify your decision.

2.2 Board Question Suggestions

1. Compare Waterfall, Prototype, Incremental, RAD and V-Model using practical industrial examples.
2. Explain why different software development projects require different SDLC models.

2.3 MCQ Focus

- Waterfall Model
- V-Model
- Prototype Model
- Incremental Model
- RAD
- RUP
- Risk Profile

3 Lecture 3 – Requirements Engineering

3.1 OBE Scenario Suggestions

1. **Amazon Healthcare Platform**

   Amazon Healthcare plans to develop an AI-powered healthcare platform for doctors, patients and hospital administrators. Each stakeholder has different expectations and several requirements conflict with one another. During development, legal policies change and customers continuously request additional features. Explain the complete Requirements Engineering lifecycle, classify the different types of requirements, identify the major challenges and recommend techniques to control requirement changes.

2. **Uber International Expansion**

   Uber expands its ride-sharing platform into a new country. Government regulations frequently change, users request new payment methods and developers experience communication gaps with stakeholders. Explain how Requirements Baseline, Scope Creep prevention, Requirement Management and Shift-Left Testing can improve the project.

3.2 Board Question Suggestions

1. Explain every phase of Requirements Engineering with suitable examples.
2. Compare Functional, Non-functional, Business, User and System Requirements using an Internet Banking System.

Mejbah Ahammad
Lecturer, Department of Data Science
American International University-Bangladesh (AIUB)
ahammadmejbah@aiub.edu

3.3 MCQ Focus

- Functional Requirements
- Non-functional Requirements
- Business Requirements
- User Requirements
- System Requirements
- Requirements Engineering Phases
- Scope Creep
- Communication Gap
- Boehm's Law
- Requirements Baseline
- Shift-Left Testing

4 Lecture 4 – Agile Software Development

4.1 OBE Scenario Suggestions

1. **Spotify Recommendation Engine**

   Spotify is developing a new AI recommendation engine where customer requirements continuously change according to user feedback. The development team releases software every two weeks and customer representatives actively participate in each iteration. Select the most suitable development methodology and explain how Agile principles, timeboxing, iterative development and human factors contribute to successful software delivery.

2. **Startup Product Development**

   A software startup plans to launch a mobile application within three months. Since customer requirements frequently change, the development team needs a flexible process that supports rapid delivery and continuous customer involvement. Explain why Agile development is more appropriate than traditional software development.

4.2 Board Question Suggestions

1. Compare Agile Development and Traditional Development using industrial examples.
2. Explain Agile Principles and discuss why iterative development improves software quality.

4.3 MCQ Focus

- Agile Principles
- Agile Assumptions
- Timeboxing
- Agile Iteration
- Human Factors
- Agile vs Traditional Development

Lecturer, Mejbah Ahammad
Department of Data Science
American International University-Bangladesh (AIUB)
ahammadmejbah@gmail.com

5 Lecture 5 – Extreme Programming (XP)

5.1 OBE Scenario Suggestions

1. **GitHub Collaborative Platform**

   GitHub develops a collaborative coding platform where customer requirements frequently change. Developers perform pair programming, continuous integration and continuous testing while customers actively participate throughout development. Explain why XP is the most appropriate methodology and discuss how XP values and practices improve software quality.

2. A software company experiences poor communication, increasing software defects and delayed releases. Explain how XP roles, values and practices can solve these problems.

5.2 Board Question Suggestions

1. Explain the complete XP lifecycle with suitable examples.
2. Discuss XP values, practices, roles and responsibilities.

5.3 MCQ Focus

- XP Values
- XP Phases
- Pair Programming
- Continuous Integration
- User Story
- Task List
- XP Roles

6 Lecture 6 – Scrum

6.1 OBE Scenario Suggestions

1. **Microsoft Teams AI Assistant**

   Microsoft introduces AI-powered meeting assistants into Microsoft Teams. Product requirements change after every Sprint based on customer feedback. Explain how Scrum roles, Product Backlog, Sprint Planning, Sprint Review and Daily Scrum help deliver high-quality software.

2. A Scrum team experiences conflicts because stakeholders continuously request new features during an active Sprint. Explain how Scrum manages changing priorities while maintaining project quality.

6.2 Board Question Suggestions

1. Explain the complete Scrum framework with roles, artifacts and ceremonies.
2. Compare Product Backlog and Sprint Backlog using an industrial example.

6.3 MCQ Focus

- Scrum Master
- Product Owner
- Scrum Team
- Sprint

Dr. Mejbah Ahammad
Department of Data Science
American International University-Bangladesh (AIUB)
mejbah@aiub.edu

- Product Backlog
- Sprint Backlog
- Sprint Planning
- Daily Scrum
- Sprint Review

7 Lecture 7 – DSDM

7.1 OBE Scenario Suggestions

1. **WHO Emergency Platform**

   The World Health Organization develops an emergency disease surveillance platform that must be delivered within strict deadlines despite changing priorities. Explain why DSDM is the most suitable methodology and discuss Timeboxing, MoSCoW prioritization, Prototyping and the 80:20 Rule.

2. A national disaster response system must be operational within four months even though several requested features cannot be completed immediately. Explain how DSDM prioritizes software functionality while meeting project deadlines.

7.2 Board Question Suggestions

1. Explain the DSDM lifecycle and its major techniques.
2. Compare Traditional Software Development and DSDM.

7.3 MCQ Focus

- DSDM Lifecycle
- Timeboxing
- MoSCoW Rules
- Flexibility
- 80:20 Rule
- Prototyping

8 Lecture 8 – Feature Driven Development (FDD)

8.1 OBE Scenario Suggestions

1. **Google Maps Feature Expansion**

   Google Maps introduces a major feature expansion involving thousands of developers across multiple countries. Explain how Feature Driven Development manages development using feature planning, feature teams, code inspections and feature-based delivery.

2. An enterprise software company plans to deliver customer-valued features every week. Explain why FDD is appropriate for this project and discuss its five development processes.

8.2 Board Question Suggestions

1. Explain the five processes of Feature Driven Development with suitable examples.
2. Discuss FDD roles and explain why code inspection is essential for software quality.

Mejbah Ahammad
Lecturer, Department of Data Science
American International University-Bangladesh (AIUB)
mejbah@aiub.edu
mejbahammad

8.3 MCQ Focus

- Feature Definition
- Feature Naming
- FDD Roles
- Five FDD Processes
- Feature Team
- Code Inspection
- Reporting
- Feature Milestones

9 Revision Checklist

- Select the most suitable SDLC model from a real-world industrial scenario.
- Identify Functional, Non-functional, Business, User and System Requirements.
- Explain the complete Requirements Engineering lifecycle.
- Compare Agile Development with Traditional Development.
- Apply XP values, practices and roles in software projects.
- Explain Scrum roles, artifacts and ceremonies.
- Apply DSDM techniques including Timeboxing, MoSCoW prioritization and the 80:20 Rule.
- Explain the Feature Driven Development lifecycle.
- Distinguish different software maintenance types.
- Identify software myths and software crisis situations.
- Justify software engineering decisions using industrial scenarios.

Lecturer, Mejbah Ahammad
American International University-Bangladesh (AIUB)
Department of Data Science
ahammadmejbah@aiub.edu